

[30] Pase: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension 총정리

저자: W. Yang, T. Li, G. Fang, H. Wei

학회/출판: SIGMOD 2020

페이지: p. 2241-2253

자료형: 데이터베이스 확장 논문

요약

Pase (PostgreSQL Approximate Similarity sEarch)는 기존 관계형 데이터베이스인 PostgreSQL에 초고차원 벡터에 대한 근사 최근접 이웃 검색 기능을 추가하는 확장이다. SIGMOD 2020에 발표된 이 논문은 전통적 RDBMS 환경에서 벡터 검색을 효율적으로 구현하는 방법을 제시한다. Pase는 PostgreSQL의 인덱싱 인프라를 활용하면서도, 초고차원 벡터의 특수성을 고려한 새로운 인덱싱 기법을 제안한다. 기존 정형 데이터와 벡터 데이터를 동시에 관리하고 쿼리하는 환경에서, 관계형 데이터베이스를 하나의 통일된 시스템으로 유지하면서 벡터 검색 성능을 확보하는 실용적 해결책을 제공한다.

상세분석

30.1 주요 문제점

PostgreSQL과 같은 기존 RDBMS에 벡터 검색을 추가할 때의 도전 과제:

- **고차원 데이터의 성능 병목:** 수천 차원의 벡터에 대해 정확한 최근접 이웃 검색은 전체 테이블 스캔을 필요로 하며, 초고 차원에서는 "차원의 저주" 현상으로 근사도 어려움
- **인덱싱 구조 부재:** B-tree, GiST 등 기존 인덱싱 기법은 고차원 공간에 부적합
- **저장소 확장:** 기존 RDBMS의 행(row) 기반 저장소에 대규모 벡터를 효율적으로 저장
- **쿼리 최적화:** 벡터 조건과 스칼라 조건을 포함한 복합 쿼리의 최적화
- **정확도-성능 트레이드오프:** 근사 알고리즘 사용 시 발생하는 정확도 손실 관리

30.2 핵심 기술: PASE의 인덱싱 기법

개요

PASE는 두 단계 인덱싱 전략을 채택:

1단계: 거칠 필터링 (Coarse Filtering)

- 벡터 공간을 여러 영역(partition)으로 분할
- 쿼리 벡터와 가까운 영역만 선택

2단계: 세밀한 재검색 (Fine Reranking)

- 선택된 영역 내에서 정확한 거리 계산
- 최상위 k개 결과 선택

분할 기법 (Partition Strategy)

균형잡힌 K-평면 분할 (Balanced K-Plane Partitioning):

```

n차원 벡터 공간
    ↓
초평면(hyperplane)으로 순차 분할
    ↓
각 영역의 크기가 대략 균등
    ↓
이진 트리 구조로 조직화
  
```

특징:

- 트리 깊이 $O(\log n)$: 로그 시간에 쿼리 벡터가 속한 영역 결정
- 메모리 효율적: 각 노드는 분할 평면만 저장
- 적응형 분할: 데이터 분포에 따른 자동 조정

세밀한 재검색 최적화

후보 확대 (Expansion Strategy):

쿼리 벡터의 영역부터 시작
↓
거리가 가까운 인접 영역으로 확대
↓
충분한 후보 수 모이면 중단

이를 통해:

- 처음부터 모든 영역을 확인하지 않음
- 쿼리 벡터 근처의 데이터에 집중
- 평균적으로 방문 영역 감소

30.3 PostgreSQL 통합 설계

확장 구조

PostgreSQL 핵심
↓
커스텀 데이터 타입: `vector(float8[], int)`
↓
인덱싱 메서드: `Pase Index`
↓
SQL 연산자: `<->` (거리), `<#>` (내적)

저장소 계층

벡터 데이터 저장:

- 고정 길이 벡터: PostgreSQL VARLENA 형식 (가변 길이)으로 저장
- 양자화: 부동소수점을 압축된 형태로 저장 가능
- 저장 효율성: 일반 float 배열 대비 메모리 절감

메타데이터와의 통합:

- 동일 테이블에 벡터와 정형 데이터 공존
- 자연스러운 JOIN 지원

예시:

```
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT,
    embedding VECTOR(1536),
    category VARCHAR(50),
    created_at TIMESTAMP
);

CREATE INDEX ON documents USING pasc (embedding);

SELECT id, content,
       embedding <-> ARRAY[...] AS distance
FROM documents
WHERE category = 'news'
ORDER BY embedding <-> ARRAY[...]
LIMIT 10;
```

30.4 성능 최적화

쿼리 계획 생성 (Query Planning)

메타데이터 통계:

- 데이터 분포 통계 수집
- 선택도 추정: 벡터 유사도 조건의 선택도 예측

조인 순서 최적화:

- 스칼라 필터 vs 벡터 필터의 순서 결정
- 선택도 낮은 조건 우선 실행

병렬 처리

벡터 검색의 병렬화:

- 인덱스 트리의 여러 서브트리를 병렬로 탐색
- 리스트 병합 알고리즘으로 결과 조합

메모리 관리

버퍼 풀 활용:

- 자주 접근하는 인덱스 노드의 캐싱
- 메모리 부족 시 LRU 제거 정책

30.5 정확도 보장

Recall 지표

근사 인덱싱의 정확도:

$$\text{Recall}@k = (\text{결과에서 실제 } k\text{-NN 중 포함된 수}) / k$$

Pase의 목표: Recall@10 = 90-95%

조절 가능한 파라미터:

- 분할 트리 깊이: 깊을수록 정확하나 느림
- 재검색 후보 확대 크기: 클수록 정확하나 느림

30.6 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 시스템이다. Pase는 다음 두 가지 측면에서 Exqutor과 관련:

1. **기존 RDBMS 통합:** Exqutor이 PostgreSQL 같은 기존 데이터베이스 위에 구축된다면, Pase 같은 벡터 확장이 백엔드 기술로 활용될 수 있다.
2. **하이브리드 검색:** Exqutor의 텍스트 필터와 벡터 유사도 검색의 조합은 Pase에서 보여주는 스칼라 조건과 벡터 조건의 결합 처리와 동일한 문제 영역이다. Pase의 쿼리 최적화 기법이 직접 적용될 수 있다.
3. **기존 시스템 활용:** Pase처럼 기존 광범위하게 사용되는 데이터베이스에 벡터 기능을 추가하는 방식은, Exqutor의 실제 배포 모델을 단순화할 수 있다.

추가 제기 문제

1. **초고차원에서의 성능:** 10,000 차원 이상의 초고차원 벡터에서 Pase의 성능이 어떻게 변하는가? 차원의 저주의 영향 정도는?
2. **K-평면 분할의 최적성:** K-평면으로 균등 분할하는 것이 모든 데이터 분포에 최적인가? 데이터 분포에 따른 적응형 분할이 성능을 향상시킬 수 있는가?
3. **인덱스 유지보수 비용:** 높은 빈도의 삽입/삭제 작업 시 인덱스 재구성의 오버헤드는? 점진적 인덱스 업데이트가 가능한가?
4. **메모리-디스크 계층:** 메모리에 모든 인덱스를 캐시할 수 없을 때, 디스크 기반 인덱싱과의 조합 전략은?
5. **정확도-성능 트레이드오프의 자동화:** 쿼리별로 요구되는 정확도 수준이 다를 때, 파라미터를 동적으로 조정하는 메커니즘은?
6. **PostgreSQL과의 호환성:** 새 버전의 PostgreSQL과의 호환성 유지 비용은? 표준 PostgreSQL의 백엔드 변경이 Pase에 미치는 영향은?